



16. Segmentation

REVIEW

외우자

Segmentation

What is segmentation?

Segmented Addressing

Segmentation Implementation

Quiz: Address Translations with Segmentation

Visual Interpretation

Segment Selection

Advantages of Segmentation

Disadvantages of Segmentation

Review

Review: Segmentation

Review: Memory Access

Physical Memory Accesses?

Conclusion

REVIEW

외우자

- $2^{10} = 1024 = K$
 - $2^{15} = 2^5 * K = 32K$
- $2^{20} = M$
- $2^{30} = G$
 - $2^{33} = 2^3 * G = 8G$
- $2^{40} = T$
- 기존에 base & bounds approach는 free space(heap과 stack 사이)라는 waste가 발생함

- 또는 Virtual memory가 physical memory보다 크면 돌리는게 불가능

Segmentation

What is segmentation?

- The idea was born in the early 1960's.
- A **segment** is just a **contiguous portion** of the address space. → 연속적이다!
- Instead of having just one base and bounds pair in MMU,
why not have a base and bounds pair per logical segment?
- A process can have three or four logically-different segments: **Code (Data), Stack and Heap**
Segment당 base & bound를 유지하자!!
- Segmentation allows the OS to place each one of those segments (Code, Stack, and Heap) in different parts of physical memory, and thus avoid filling physical memory with unused virtual address space.
- Each segment can independently
 - be placed separately in physical memory
 - grow and shrink
 - be protected (separate read/write/execute protection bits)

Segmented Addressing

- Process now specifies segment and offset within segment
- How does **process designate a particular segment?**
 - Use **part of logical address**
 - Top bits of logical address select segment
 - Low bits of logical address select offset within segment

Segmentation Implementation

- MMU contains Segment Table (per process)
 - Each segment has own base and bounds, protection bits
 - Example: 14 bit logical address, 4 segments → 2개의 bit로 segment 구분
 - how many bits for segment? How many bits for offset?
- $16K = 2^{14}$
- 각 segment의 size는 $2^{12} = 4K(0xfff) \Rightarrow$ bounds 최댓값

Segment	Base	Bounds	R	W
0	0x2000	0x6ff	1	0
1	0x0000	0x4ff	1	1
2	0x3000	0xfff	1	1
3	0x0000	0x000	0	0

- base랑 bound는 실제 물리적인 주소

Quiz: Address Translations with Segmentation

Translate logical addresses (in hex) to physical addresses

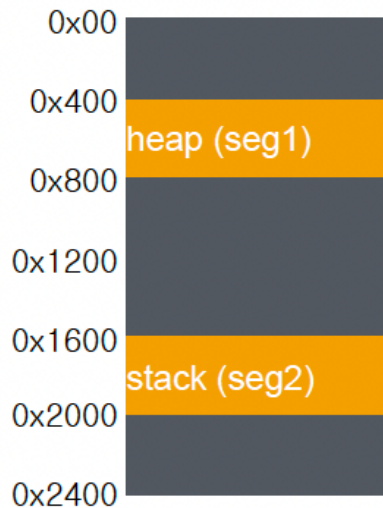
0x0240:

0x1108: 0001 0001 0000 1000

0x265c:

0x3002:

Visual Interpretation



Segment numbers

0: code+data

1: heap

2: stack

Virtual (hex)	Physical
load 0x2010, R1	load 0x1610, R1
load 0x1010, R1	load 0x410, R1
load 0x1100, R1	load 0x500, R1

0x2010: 0010 0000 0001 0000 ⇒ 2: stack | offset: 0x0010

0x1010: 0001 0000 0001 0000 ⇒ 1: heap | offset: 0x10

0x1100: 0001 0001 0000 0000 ⇒ 1: heap | offset: 0x100

Segment Selection

```
// get top 2 bits of 14-bit VA
Segment = (VirtualAddress & SEG_MASK) >> SEG_SHIFT
// now get offset
Offset = VirtualAddress & OFFSET_MASK
if(Offset >= Bounds[Segment])
    RaiseException(PROTECTION_FAULT)
```

else

PhysAddr = Base[Segment] + Offset

Register = AccessMemory(PhysAddr)

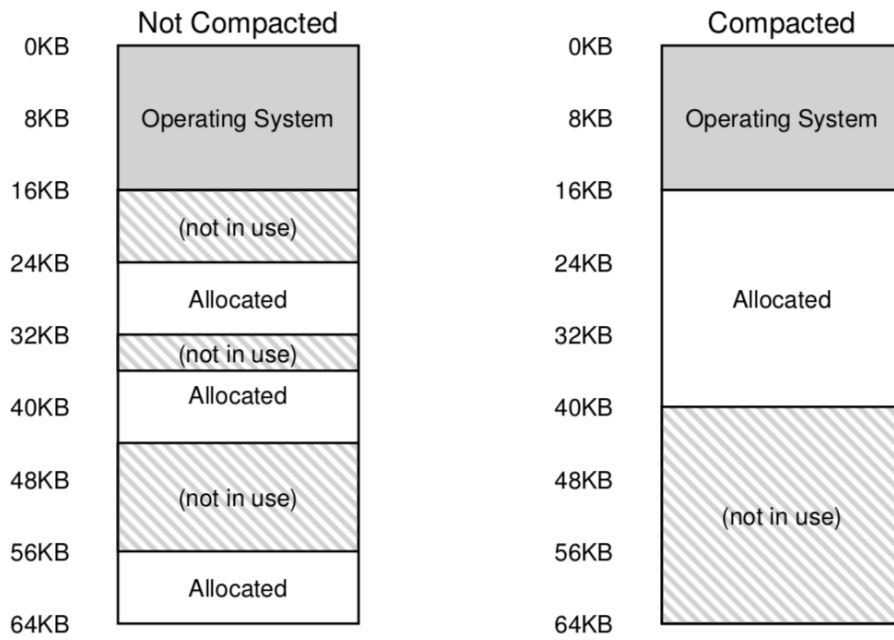
- SEG_MASK = 0x3000(110000000000000)
- SEG_SHIFT = 12
- OFFSET_MASK = 0xFFF (00111111111111)

Advantages of Segmentation

- Enables **sparse allocation of address space**
 - Stack and heap **can grow independently**
 - Heap: If no data on free list, **dynamic memory allocator** requests more from OS (e.g., UNIX: malloc calls sbrk())
 - Stack: **OS** recognizes reference outside legal segment, extends stack implicitly
- Different **protection** for different segments
 - Read-only status for code
- Enables **sharing** of selected segments
- Supports **dynamic relocation** of each segment

Disadvantages of Segmentation

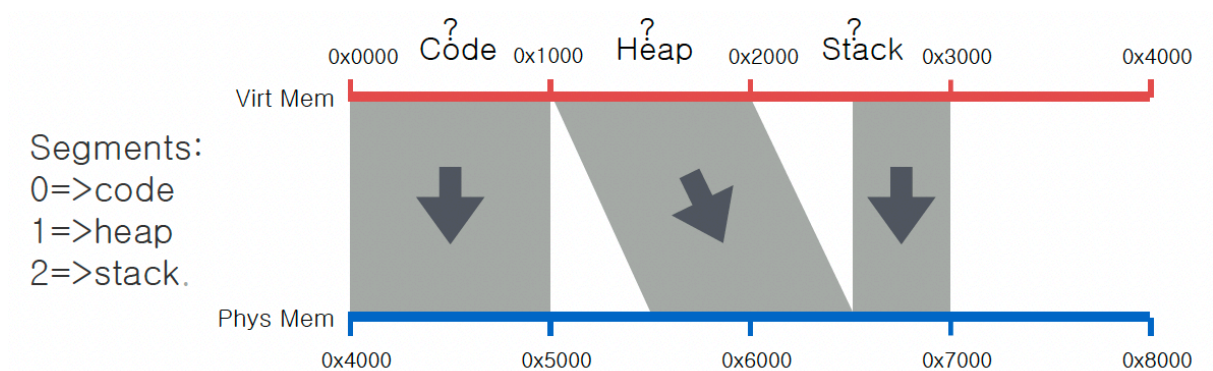
- Each segment must be allocated contiguously
 - May not have sufficient physical memory for large segments, resulting in **external fragmentation**
- Fix in next lecture with paging



→ 이렇게 compaction을 할 수도 있긴함

Review

Review: Segmentation



Seg	Base	Bounds
0 (code)	0x4000	0xfff
1 (heap)	0x5800	0xfff
2 (stack)	0x6800	0x7ff

Review: Memory Access

모든 주소 계산은 하드웨어적으로 구현?

```
0x0010: movl 0x1100, %edi
0x0013: addl $0x3, %edi
0x0019: movl %edi, 0x1100
```

%rip: 0x0010

Seg	Base	Bounds
0 (code)	0x4000	0xfff
1 (heap)	0x5800	0xfff
2 (stack)	0x6800	0x7ff

Physical Memory Accesses?

- Fetch instruction at logical address 0x0010** → Physical address: $0x4000 + 0x0010 = 0x4010$
 - exec, load from logical address 0x1100 → Physical address: $0x5800 + 0x0100 = 0x5900$
- Fetch instruction at logical address 0x0013** → Physical address: $0x4000 + 0x0013 = 0x4013$
 - exec, no load
- Fetch instruction at logical address 0x0019** → Physical address: $0x4000 + 0x0019 = 0x4019$
 - Exec, store to logical address 0x1100 → Physical address: $0x5800 + 0x0100 = 0x5900$

Conclusion

- HW+OS work together to virtualize memory
 - Give illusion of private address space to each process
- Add MMU registers for base+bounds so translation is fast

- OS not involved with every address translation, only on context switch or errors
- Dynamic relocation with segments is good building block
 - Next lecture: Solve fragmentation with paging